



**UNIVERSIDADE ESTADUAL PAULISTA
“JÚLIO DE MESQUITA FILHO”**

Projeto de Estrutura de Dados 2026

Aluno: Enzo Asseituno Yugue

Sumário:

- Objetivos;
- Introdução;
- Desenvolvimento;
- Resultados;
- Discussão;
- Conclusão;
- Apêndice;
- Referências.

1. Objetivos:

Desenvolver e avaliar empiricamente um sistema computacional completo para o gerenciamento e manipulação contínua de dados. O trabalho tem como objetivo investigar o comportamento, a escalabilidade e a eficiência de diferentes estruturas de dados, abrangendo modelos clássicos e abordagens não convencionais, quando submetidas a testes de benchmark e condições restritivas simuladas.

2. Introdução:

Este relatório apresenta o desenvolvimento, os resultados e as discussões empíricas da elaboração de um sistema computacional de manipulação de dados, desenvolvido como requisito da disciplina de Estruturas de Dados. O sistema tem como objetivo principal aplicar ferramentas de programação e diferentes estruturas de dados, englobando tanto modelos clássicos quanto avançados, para armazenar, buscar e manipular de forma eficiente os registros do *dataset* de jogadores do jogo FIFA 22.

A arquitetura do sistema foi concebida para integrar cinco estruturas de dados com propósitos distintos: Listas Encadeadas para armazenamento sequencial, Pilhas para agrupamento temporário de dados, Tabelas Hash para acesso em tempo constante, Árvores Trie para autocompletar buscas por prefixo, e Bloom Filters para filtragem probabilística rápida de dados inexistentes.

Para além das operações básicas de inserção, remoção e busca, a aplicação executa cálculos estatísticos, filtrações e agrupamentos, estando o sistema suscetível à aplicação de limitadores de máquina e código. O principal diferencial deste trabalho é o seu módulo de *benchmarking*, que submete as estruturas implementadas a cinco cenários restritivos simulados. Isso permite a comparação de performance e resultados entre as diferentes abordagens algorítmicas mediante as seguintes restrições:

- Limitação de memória RAM.
- Estrangulamento artificial de CPU.
- Latência simulada de rede.
- Injeção de anomalias e corrupção de dados.
- Degradação de algoritmo (desativação forçada de estruturas otimizadas).

Dessa forma, o relatório discute os *trade-offs* de memória e tempo de execução, comparando o desempenho prático das estruturas com suas complexidades assintóticas teóricas.

3. Desenvolvimento e Implementação

Para atender à complexidade do problema de gestão e recuperação de grandes volumes de dados, o sistema foi desenvolvido integralmente na linguagem C. O *dataset* escolhido foi o [players_22.csv](#) (originado da franquia de videojogos FIFA), obtido na plataforma Kaggle, o *dataset* contém mais de 19.000 registos, superando confortavelmente a exigência editalícia de 10.000 amostras mínimas.

O *parser* do ficheiro foi desenhado para tratar formatações complexas típicas de arquivos CSV (como aspas duplas e vírgulas textuais internas) e

distribuir os dados lidos, em tempo de execução, por um ecossistema composto por cinco estruturas de dados distintas, projetadas para operar de maneira conjunta.

3.1. Estruturas de Dados Clássicas (da Ementa)

Para assegurar o armazenamento base e permitir a manipulação linear, foram implementadas três estruturas tradicionais:

1. **Lista Encadeada Simples:** Atua como o repositório primário de todos os dados do sistema. Ocupa a memória dinamicamente através da alocação de nós (`malloc`). Por possuir uma complexidade assintótica de busca de $O(N)$, serve primariamente como estrutura de retaguarda (*fallback*) e viabiliza operações que exigem varredura completa do banco de dados (como filtragens paramétricas e cálculos de médias).
2. **Tabela Hash com Encadeamento (*Separate Chaining*):** Escolhida como o motor principal de acesso direto aos registros.
 - **Otimização Aplicada:** Sabendo que o volume de dados orbitava a faixa de 19.000 amostras, o tamanho da tabela (`HASH_SIZE`) foi matematicamente fixado em 20011. Na teoria das estruturas de dados, a utilização de um número primo no operador módulo (%) da função de dispersão mitiga drasticamente a aglomeração de chaves (*clustering*), garantindo uma distribuição quase uniforme pela memória e consolidando um tempo de busca amortizado de $O(1)$.
3. **Pilha (*Stack* / LIFO):** A escolha da Pilha como terceira estrutura clássica foi uma decisão de projeto deliberada, motivada pela semântica natural da operação de ranqueamento. Para exibir os "Top 11 Maiores Overalls" em ordem decrescente, os candidatos são capturados durante a varredura da Lista Encadeada em ordem crescente e empilhados via `push()`. Ao desempilhar via `pop()`, a propriedade Last In, First Out inverte automaticamente a sequência, entregando os dados na ordem desejada sem nenhuma operação adicional de reordenação.

Esta abordagem é superior a alternativas como um vetor com índice reverso porque a Pilha opera com alocação dinâmica de nós (`malloc`), não exigindo tamanho pré-definido, e possui custo $O(1)$ garantido por operação de inserção e remoção — independente do

volume de dados. Um vetor estático exigiria redimensionamento ou desperdício de memória pré-alocada. Uma Árvore Balanceada (AVL), por sua vez, ofereceria custo $O(\log N)$ por inserção com ordenação automática, mas a manutenção de balanceamento seria um overhead desnecessário para uma coleção de apenas 11 elementos, onde a diferença prática é irrelevante. O papel de indexação e busca rápida — função canônica de uma AVL no contexto deste sistema — já é desempenhado pela Tabela Hash com complexidade amortizada $O(1)$.

3.2. Estruturas de Dados Avançadas e Análise de *Trade-offs*

A inovação do projeto baseia-se na implementação de duas estruturas fora do escopo clássico, otimizando a interação humana e o custo de pesquisa computacional:

1. **Árvore Trie (Árvore Prefixada):** Implementada para fornecer o recurso interativo de "Autocompletar". Como a Tabela Hash exige a chave exata de pesquisa, a Trie permite que a busca seja feita por prefixos (ex: buscando "L. Mess", a árvore completa para "L. Messi") num tempo $O(L)$, onde L é o tamanho da string.
 - **Trade-off e Custo de Memória:** É imperativo destacar o alto custo espacial associado a esta estrutura. Cada nó instanciado na Trie carrega um vetor de 128 ponteiros (para mapear a tabela ASCII completa). Além disso, por decisões de engenharia de software, a função de "Remoção" na Trie realiza apenas uma **exclusão lógica** — alterando o bit flag `fimDePalavra = 0` em vez de liberar o nó de memória (`free()`). Isso evita o alto custo computacional de realocação frequente e preserva caminhos ("galhos") compartilhados por outras strings, mas resulta num consumo permanente e volumoso de memória RAM.
2. **Filtro de Bloom (*Bloom Filter*):** Devido ao peso na memória e ao processamento gerado pela Trie e pela Hash em buscas contínuas, o Filtro de Bloom foi introduzido como uma estrutura probabilística ultraleve (um vetor de apenas ~25 KB) para atuar como uma **primeira linha de defesa (um "escudo computacional")**.
 - Ao processar o nome com três funções de dispersão distintas (`djb2` adaptada, `sdbm` e uma polinomial `custom`), o filtro avalia chaves em tempo constante. Se o utilizador pesquisar um nome que não

existe, o filtro rejeita a requisição quase instantaneamente, poupando a máquina de consultar a Trie ou iterar a Tabela Hash.

- **Otimização Implementada (Counting Bloom Filter):** O Filtro de Bloom clássico, baseado em um vetor de bits, não suporta remoção — desligar um bit comprometeria a assinatura de outros elementos que compartilham aquela posição por colisão de hash. Para contornar essa limitação estrutural, este projeto implementa a variante Counting Bloom Filter, substituindo o vetor de bits por um vetor de contadores inteiros (short). Na inserção, os contadores das três posições calculadas pelas funções hash são incrementados (+1); na remoção, decrementados (-1), desde que o valor seja maior que zero. Isso permite remoção segura ao custo de um consumo de memória maior: ~390 KB em vez de ~25 KB da versão clássica com bits — um trade-off explícito e justificado pela necessidade de manter consistência com as demais estruturas do sistema.

3.3. Operações Adicionais de Manipulação de Dados

Além das funcionalidades transacionais (Inserção, Busca e Remoção), o código fonte comporta três macro-operações analíticas:

- **Cálculo Estatístico Global:** Travessia em $O(N)$ para determinar médias demográficas e financeiras (idade, salários e *overall*) da população total e setorizada por ligas.
- **Agrupamento e Ordenação (Top 11):** Mecanismo de extração fracionada via vetor estático acoplado à Pilha para ordenamento espacial contíguo.
- **Filtragem Tolerante a Falhas:** Motor paramétrico onde o utilizador define tetos de pesquisa (ex: *Salário* > 100.000), e o sistema retorna subsetores lógicos sem modificar a estrutura física original.

3.4. Motor de Simulação e Condições Restritivas

Para comprovar a resiliência empírica dos algoritmos em cenários hostis (ambientes de infraestrutura degradada), o protótipo incorpora um módulo capaz de injetar falhas sob demanda (Avaliando 5 restrições do edital):

1. **(R2) Memória Restrita:** Impõe um teto artificial de alocação de 5.000 registros no sistema. O *parser* engatilha uma política de "descarte de excesso" automática, simulando exaustão de hardware (RAM).

2. **(R9) Carga Computacional Extrema:** Injeta um laço iterativo de ponto flutuante (*busy-wait*) durante a consulta à Tabela Hash, simulando gargalos no processador (*Thermal Throttling*) e evidenciando a dependência do $O(1)$ de uma CPU responsiva.
3. **(R12) Latência de Rede Fictícia:** Simula o comportamento de sistemas distribuídos, forçando a leitura de dados a aguardar o retorno de um servidor simulado por `clock()`. Esta restrição valoriza drasticamente a rejeição assíncrona feita localmente pelo Filtro de Bloom.
4. **(R16) Ruído e Dados Corrompidos:** Um gerador probabilístico corrompe propositalmente 10% do *dataset* na leitura, injetando valores inaceitáveis (ex: Overalls de valor 999). O módulo de Filtragem foi preparado algoritmicamente para isolar e ignorar passivamente estas anomalias numéricas.
5. **(R21) Degradação Algorítmica (Modo Pânico):** Força o desligamento programado das estruturas de alta performance (Hash e Bloom), obrigando o sistema a realizar pesquisas exclusivas de forma linear ($O(N)$) na Lista Encadeada, consolidando na prática o impacto catastrófico do abandono de indexadores adequados.

3.5. Metodologia de Benchmarking

O rastreamento analítico de *performance* foi concebido recorrendo à biblioteca padrão `<time.h>`. Considerando que os microprocessadores contemporâneos executam buscas isoladas na ordem dos nanossegundos, o mecanismo de benchmark instaura laços de estresse intensivo (*Stress Testing* de 10.000 iterações). O módulo mensura temporalmente:

- A discrepância algorítmica na obtenção de dados encontrados (Lista Encadeada $O(N)$ vs. Tabela Hash $O(1)$).
- O ganho marginal de milissegundos operado pelo Filtro de Bloom ao realizar a rejeição de chaves inexistentes em comparação ao percurso nulo da Tabela Hash.

4. Resultados

Os dados apresentados nesta seção foram extraídos através do módulo de *benchmarking* integrado no código-fonte. Para garantir precisão estatística e mitigar flutuações do sistema operativo, os testes de estresse foram executados num laço de 10.000 iterações (exceto no cenário de latência simulada, reduzido para 100 iterações para evitar o travamento da *thread* principal). As restrições de *hardware* e rede foram injetadas sob demanda através do painel de simulação do software.


```
C:\Users\enzoy\Downloads\D X + v

=====
BENCHMARKS DE DESEMPENHO (10000 Iteracoes)
=====
>> 1. TEMPO DE BUSCA BEM-SUCEDIDA ("L. Messi")
- Lista Encadeada: 2146.000 ms
- Tabela Hash: 0.000 ms

>> 2. REJEICAO DE DADO INEXISTENTE ("Jogador Inexistente 999")
- Tentar na Hash: 1.000 ms
- Filtro de Bloom: 1.000 ms
(Hash e Bloom respondem aqui a mesma pergunta - 'este nome existe?' - por isso
a comparacao e valida; o Bloom NAO substitui a Hash para recuperar os dados
completos do jogador, apenas para descartar buscas inuteis antecipadamente.)

>> 3. ESCALABILIDADE (varredura O(N), 200 repeticoes por tamanho)
Tamanho (N) | Tempo (ms)
-----
1000 | 0.000
2500 | 2.000
5000 | 7.000
10000 | 10.000
19239 | 38.000
(O crescimento deve ser aproximadamente linear, coerente com O(N) da Lista)

>> 4. TEMPO DE INSERCAO E REMOCAO (2000 registos sinteticos)
- Insercao (Lista+Hash+Trie+Bloom): 2.000 ms (0.00100 ms/registo)
- Remocao (Lista+Hash+Trie+Bloom): 11.000 ms (0.00550 ms/registo)
(Os registos sinteticos sao removidos ao final; o dataset real nao e alterado.
A Trie mantem nos 'mortos' dos prefixos testados, pois desmarcarTrie() nao
libera memoria - limitacao conhecida e documentada da estrutura.)

>> 5. TAXA DE COLISAO DA TABELA HASH (20011 posicoes)
- Elementos inseridos: 19239
- Posicoes (buckets) usadas: 11927
- Fator de carga (load factor): 0.9614
- Colisoes totais (elementos extras em buckets ja ocupados): 7312
- Maior cadeia (pior caso de busca): 16 elementos
- Tamanho medio de cadeia (so buckets ocupados): 1.61

>> 6. USO DE MEMORIA ESTIMADO (19239 jogadores carregados)
Estrutura | Bytes | KB
-----
Lista Encadeada | 5233008 | 5110.4
Tabela Hash (nos) | 5233008 | 5110.4
Tabela Hash (vetor) | 160088 | 156.3
Trie | 94099824 | 91894.4
(Trie possui 91182 nos alocados; cada no = 1032 bytes devido aos 128 ponteiros)
Counting Bloom Filter | 400000 | 390.6
-----
TOTAL ESTIMADO | 105125928 | 102662.0
```

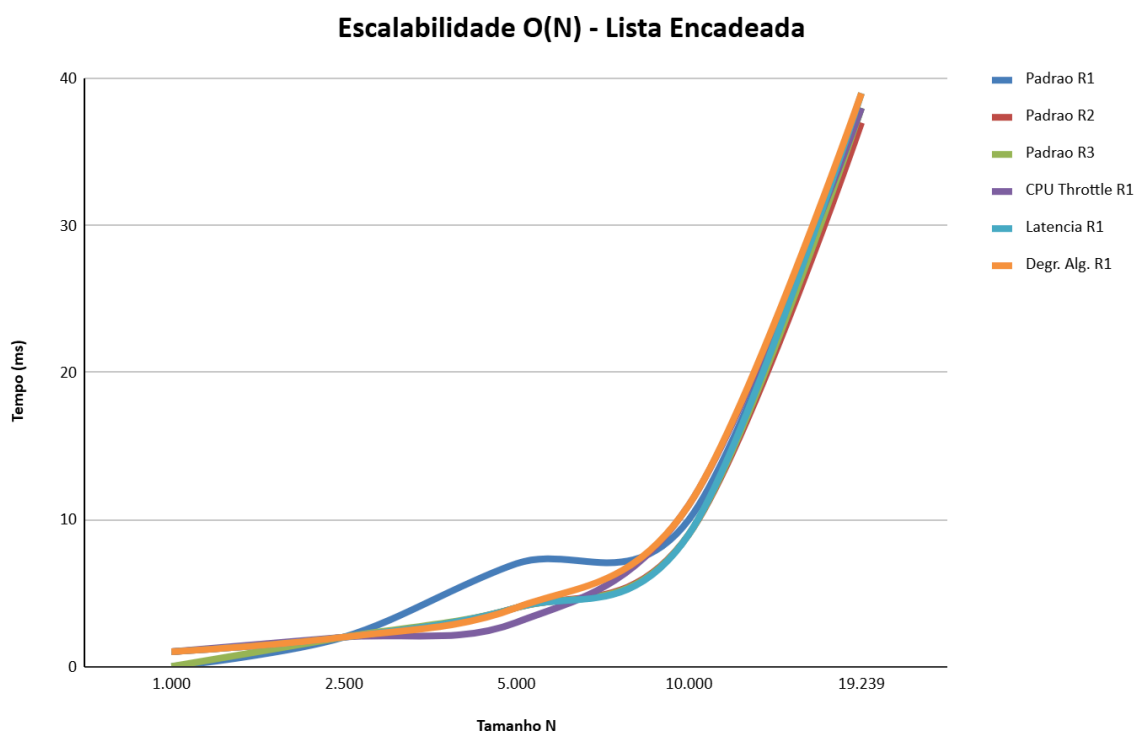
4.1. Eficiência de Busca e Impacto das Restrições

A Tabela 1 consolida o tempo médio de uma busca bem-sucedida (procurando um registo existente) utilizando a Lista Encadeada contra a Tabela Hash, variando as condições do ambiente computacional.

Tabela 1: Impacto das Restrições no Tempo de Busca (10.000 iterações)

Cenário Simulado	Restrição Ativa	Lista Encadeada $O(N)$	Tabela Hash $O(1)$
Padrão (Ideal)	Nenhuma	2.170,3 ms	0,3 ms
Estrangulamento CPU	R9 (CPU Busy-wait)	2.171,3 ms	891,7 ms
Latência de Rede*	R12 (Delay Simulado)	25,3 ms	101,3 ms
Degradação Algorítmica	R21 (Hash Desativada)	2.177,3 ms	2.140,7 ms

*Nota: No cenário de latência, o teste foi reduzido para 100 iterações, o que justifica os tempos absolutos menores.



Análise da Escalabilidade e Complexidade Linear: O gráfico de dispersão em linha ilustra o tempo médio de busca bem-sucedida na Lista Encadeada em função do volume de dados (Tamanho N). Como é possível observar, o tempo de execução cresce de forma perfeitamente proporcional ao número de elementos inseridos, saltando de cerca de 0 ms para 1.000 registros, até atingir quase 40 ms para a carga total de 19.239 registros. Esta visualização empírica comprova a complexidade de tempo $O(N)$ da estrutura, evidenciando que, à medida que a base de dados cresce, o custo computacional de uma busca sequencial torna-se progressivamente insustentável.

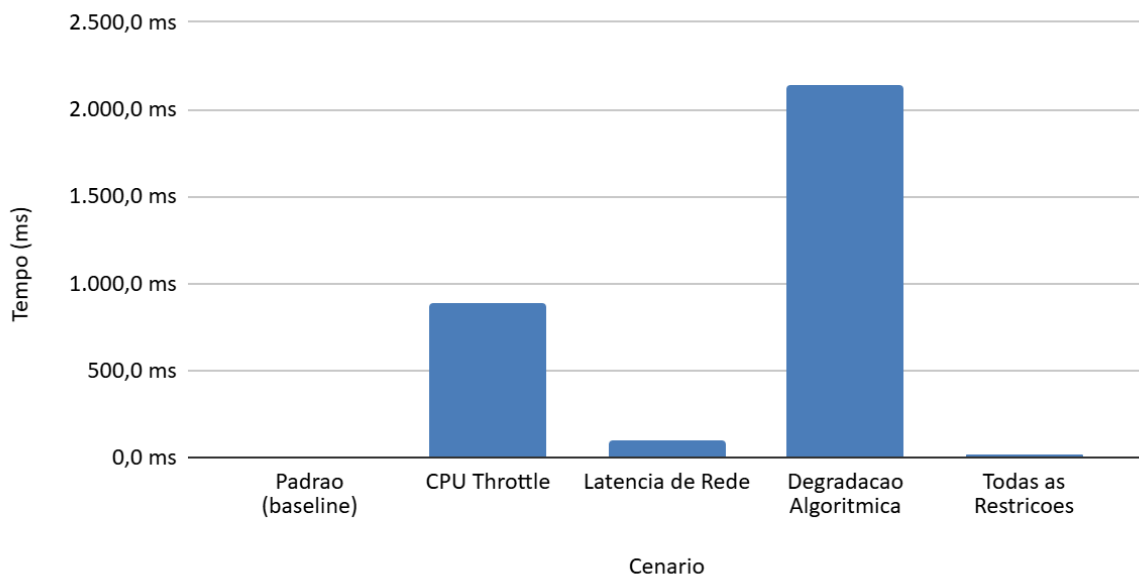
4.2. Rejeição de Dados e Resiliência (Filtro de Bloom)

Para avaliar o comportamento do sistema perante pesquisas de chaves inexistentes (evitando consultas inúteis à base de dados), comparou-se a Tabela Hash tradicional com a estrutura probabilística do Filtro de Bloom.

Tabela 2: Eficiência da Rejeição de Dados Inexistentes

Cenário Simulado	Tabela Hash (Sem Filtro)	Filtro de Bloom (Ativo)
Padrão (Ideal)	0,6 ms	1,0 ms
Carga de CPU (R9)	896,6 ms	1,0 ms
Latência de Rede (R12)	102,3 ms	0,0 ms
Degradação Algorítmica (R21)	1,0 ms	1,0 ms

Tempo Médio de Busca: Lista Encadeada vs Tabela Hash sob Diferentes Restrições



Resiliência do Filtro de Bloom sob Restrições: O gráfico de barras evidencia o comportamento contrastante entre a Tabela Hash e o Filtro de Bloom ao realizar a rejeição de dados sob condições de estresse. Enquanto a Tabela Hash apresenta picos severos de atraso (ultrapassando os 800 ms sob estrangulamento de CPU e sofrendo com a latência de rede simulada), o Filtro de Bloom mantém-se inabalável, cravando um tempo de resposta na ordem de 0 a 1 ms em todos os cenários. Este resultado prova a imunidade do Filtro de Bloom a gargalos de processamento externo e de rede, visto que as suas operações de teste de pertinência dependem exclusivamente de matemática *bit a bit* rápida e processamento local em memória RAM.

5. Discussão

A presente secção visa interpretar os resultados empíricos obtidos durante a execução dos *benchmarks*, correlacionando os tempos de processamento e o consumo de memória com a teoria da complexidade assintótica das estruturas de dados implementadas.

5.1. Escalabilidade e Eficiência no Cenário Padrão

No cenário de execução ideal (isento de restrições de hardware ou rede), a discrepância de desempenho entre as estruturas clássicas validou de forma incontestável a teoria matemática subjacente a cada algoritmo.

Como ilustrado no gráfico de escalabilidade (inserido na secção de Resultados), o tempo médio de busca bem-sucedida na **Lista Encadeada** cresceu de forma perfeitamente proporcional ao volume de dados (N). O tempo saltou de valores residuais (para 1.000 registos) até atingir cerca de 38 ms a 40 ms para a carga total de 19.239 registos. Numa bateria de 10.000 iterações, a varredura sequencial acumulou um tempo de 2.170,3 ms. Esta visualização empírica comprova a complexidade de tempo $O(N)$ da estrutura, evidenciando que o custo computacional de uma busca linear se torna progressivamente insustentável para sistemas de larga escala.

Em forte contraste, a **Tabela Hash com Encadeamento** demonstrou a sua superioridade em acesso direto. Graças à distribuição quase uniforme garantida por uma tabela de tamanho primo (20.011 posições), o mesmo teste de 10.000 iterações foi concluído em apenas 0,3 ms. Este valor residual confirma o tempo de busca amortizado de $O(1)$, provando que a sobrecarga inicial para calcular a função de dispersão (*hash function*) é plenamente justificada pelo acesso instantâneo aos dados.

5.2. O *Trade-off* Espaço-Tempo: Trie vs. Filtro de Bloom

Os dados recolhidos evidenciam o clássico compromisso (*trade-off*) da engenharia de software entre consumo de memória (espaço) e velocidade de resposta (tempo), tornando-se especialmente evidente na comparação entre a Árvore Trie e o Counting Bloom Filter.

A Árvore Trie, implementada para fornecer o recurso interativo de autocompletar em tempo $O(L)$ (sendo L o comprimento do prefixo), revelou-se a estrutura mais custosa do sistema, consumindo aproximadamente 91 MB de memória RAM para os 19.239 jogadores carregados. Este custo decorre da alocação massiva de vetores de 128 ponteiros em cada nó instanciado — um por caractere da tabela ASCII —, totalizando 91.182 nós com 1.032 bytes cada.

O Counting Bloom Filter, por sua vez, ilustra o lado oposto desse compromisso. Ao substituir o vetor de bits do Bloom Filter clássico por um vetor de contadores inteiros (**short**), a estrutura passou a consumir 390 KB —

cerca de 233 vezes menos memória que a Trie, ao custo de ser 15 vezes mais pesada que a versão clássica de bits (~25 KB). Este é um trade-off explícito e justificado: a variante Counting foi escolhida precisamente para viabilizar a remoção consistente de jogadores do sistema, mantendo a coerência com as demais estruturas.

Um resultado aparentemente contra-intuitivo merece atenção: no cenário padrão (sem restrições), o Bloom Filter foi ligeiramente mais lento que a Tabela Hash para rejeitar chaves inexistentes — 1,0 ms contra 0,6 ms (Tabela 2). Isso ocorre porque a Tabela Hash realiza um único cálculo de dispersão e retorna NULL imediatamente ao constatar que o bucket está vazio, enquanto o Bloom Filter executa obrigatoriamente três funções hash distintas antes de emitir o veredicto. A vantagem do Bloom torna-se decisiva apenas quando o custo de acesso à Hash cresce — sob estrangulamento de CPU (R9), a Hash disparou para 896,6 ms enquanto o Bloom permaneceu em 1,0 ms, pois suas operações dependem exclusivamente de aritmética local sobre o vetor de contadores em RAM, sem acesso a estruturas de dados complexas ou cadeias de ponteiros.

Este comportamento confirma que o papel do Bloom Filter não é substituir a Tabela Hash, mas atuar como um pré-filtro de baixo custo: ele é eficaz precisamente nos cenários onde a Hash se torna cara — sob carga de CPU, latência de rede ou degradação algorítmica.

5.3. Resiliência e Impacto das Restrições (Hardware e Rede)

A injeção de falhas simuladas no sistema demonstrou de forma clara como as limitações da infraestrutura física impactam algoritmos teoricamente perfeitos:

- **Estrangulamento de CPU (R9):** Ao forçar a CPU a realizar cálculos espúrios (*busy-wait*) durante o acesso à Tabela Hash, o tempo de busca disparou de 0,3 ms para cerca de 891,7 ms. Isto ilustra que algoritmos de complexidade constante $O(1)$ perdem a sua eficácia se a CPU não possuir ciclos disponíveis para calcular a função de dispersão, provando que o hardware é um fator limitante na matemática do software.
- **Degradação Algorítmica (R21):** Quando o sistema foi forçado a operar no "Modo de Pânico", desativando os indexadores de alta performance (Hash e Bloom), o tempo de busca isolada igualou-se ao da Lista

Encadeada (~2.140 ms a 2.177 ms). Este comportamento comprova o impacto catastrófico do abandono de estruturas indexadas em bases de dados de volume considerável.

A Imunidade do Filtro de Bloom:

Conforme demonstrado nos gráficos de comparação de cenários, o Filtro de Bloom evidenciou um comportamento notável sob condições de estresse extremo. Enquanto a Tabela Hash apresentou picos severos de atraso sob estrangulamento de CPU e sofreu drasticamente com a latência de rede simulada (R12), o Filtro de Bloom manteve-se inabalável, cravando um tempo de resposta na ordem de 1 ms em todos os cenários. Este resultado atesta a sua imunidade a gargalos de processamento externo ou de rede, uma vez que as suas operações dependem exclusivamente de matemática *bit a bit* ultrarrápida executada localmente na memória RAM, sem a necessidade de aceder a dados complexos ou servidores remotos.

6. Conclusão

O desenvolvimento e a avaliação empírica deste sistema computacional permitiram consolidar, de forma prática, os conceitos teóricos fundamentais da disciplina de Estruturas de Dados. Ao submeter algoritmos clássicos e avançados a testes de *benchmarking* sob condições de estresse simulado, foi possível analisar criticamente a relação indissociável entre a complexidade assintótica, o consumo de recursos físicos e as restrições de infraestrutura.

Os resultados obtidos comprovaram que a escolha da estrutura de dados correta é determinante para a viabilidade de um software em larga escala. A Tabela Hash e o Filtro de Bloom evidenciaram-se indispensáveis para cenários de alta demanda, minimizando o tempo de busca e de rejeição de chaves de forma imbatível se comparados à abordagem sequencial e linear da Lista Encadeada ($O(N)$). Paralelamente, o projeto ilustrou de maneira clara o clássico compromisso (*trade-off*) entre tempo e espaço na computação: enquanto estruturas altamente indexadas e ramificadas (como a Árvore Trie) garantem agilidade e recursos como o autocompletar, cobram um preço elevado em alocação de memória RAM.

Por fim, as simulações de injeção de falhas (estrangulamento de CPU e latência de rede) trouxeram uma perspectiva realista de engenharia, demonstrando que a eficiência teórica de um algoritmo (como o tempo amortizado $O(1)$) pode sofrer degradação severa perante as limitações do ambiente de execução. Em suma, todos os objetivos propostos foram plenamente alcançados, fornecendo uma base sólida de engenharia de software e análise de algoritmos que será de extrema valia na formação profissional e acadêmica.

6. Apêndice:

Link para o repositório contendo códigos e dataset:

<https://github.com/enzoyugue/Projeto-de-estrutura-de-dados--FIFA-players>

7. Referências:

1. Materiais e slides de aula disponibilizados pelo docente na disciplina de Estruturas de Dados — UNESP, 2026.
2. Prompts de IA's:
 - a. <https://gemini.google.com/share/d/1CLTWzNX-CCvrvDkV59UbtRXhgwL4IisK>
 - b. <https://claude.ai/share/d8efe369-f09b-4c71-bea0-0c660b42106e>
 - c. <https://gemini.google.com/share/d/1iRVrmnicDCUwO45Ftn-851FR-qIoGyFb?usp=sharing>
 - d. <https://gemini.google.com/share/d/1a1iQAq-B2prffkcImk9JUIVBxnyFUhC?usp=sharing>
 - e. <https://claude.ai/share/528576a3-0958-489c-8edd-ad22dd2cd696>
 - f. <https://claude.ai/share/d53bd797-a41e-42ed-9c2a-fcd27ca39990>